CHAKRAVYUH 2026 | VIT-AP University

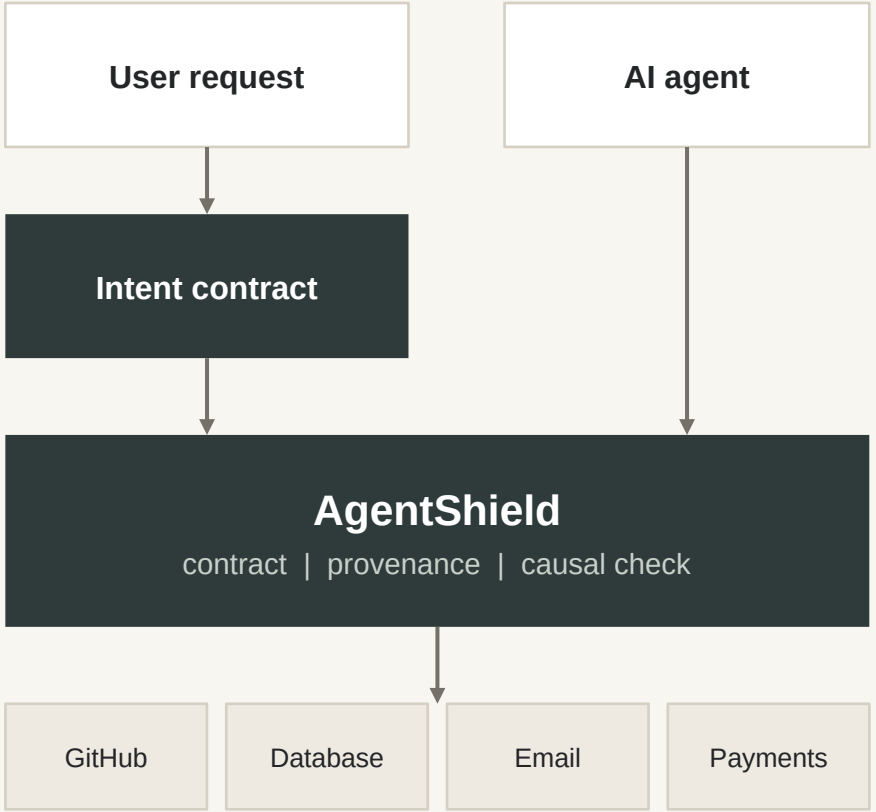
AgentShield

Intent-bound, provenance-aware execution control for AI agents

The agent can think anything. It can only do what the user asked for.

Problem statement: CB-05 **Track:** Cybersecurity & Blockchain

Team:	`teamCookie()`	
Team leader:	Kathrina Elangbam	24BCE7264
Members:	Shaunak Choudhury	24BCE8098
	Pranjal Kumar	24BCE7018
	Rakesh Kumar Rabha	24BCC7002



Agents act on what they read, and attackers write what they read

Agents now hold real credentials: GitHub tokens, database keys, email, payments, a shell.

An LLM has no separation between control and data. The user's instructions and every tool result share one context window.

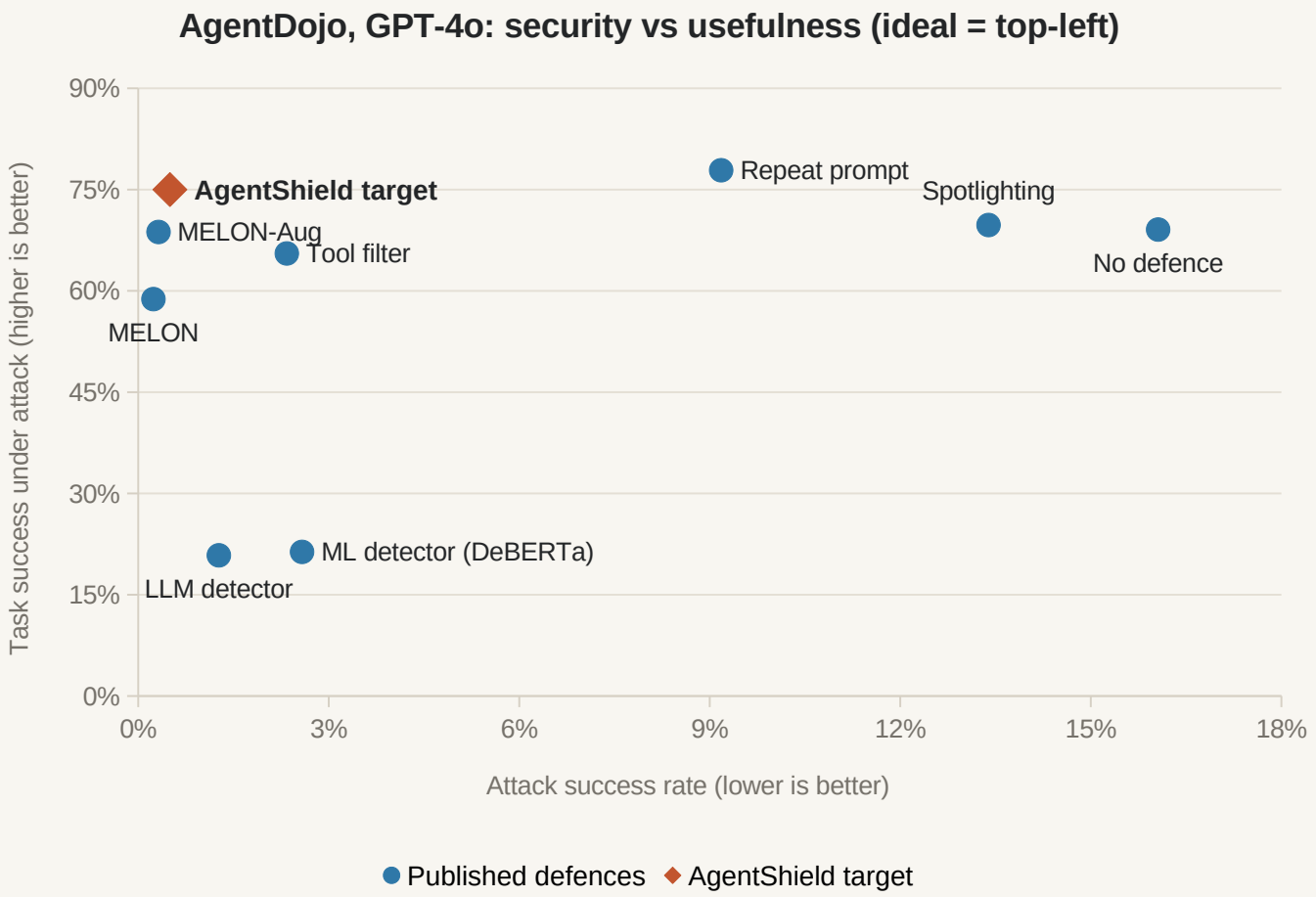
So text an attacker plants in an issue, a ticket or a web page can steer the agent. This is indirect prompt injection, a classic confused-deputy attack.

The model is not the only weak point. The missing checkpoint between what the agent decides and what it does is.



Pattern of the GitHub MCP exploit disclosed by Invariant Labs, May 2025. Repo names are illustrative.

Research shows today's defences break, or break the agent



Detectors get bypassed

Adaptive attacks beat 12 published defences, most at over 90% success. Human red-teamers broke every one they were given.

Strong defences cost usefulness

MELON: task success 80% to 68%. CaMeL: 84% to 77% with about 2.8x the tokens. An independent Progent replication: 45% to 27%.

Argument provenance is still open

Policy checkers see tool names and arguments, but not where those values came from. A 2026 study calls this an open design problem.

Chart data: MELON paper (Zhu et al., ICML 2025), AgentDojo, GPT-4o, averaged over 4 attacks. Findings: Nasr, Carlini et al. (USENIX Security 2026); CaMeL (SaTML 2026); Narisetty et al. (arXiv 2606.26479, 2026). The AgentShield point is a design target, not a measured result.

Where existing solutions sit

Approach	Drop-in (no agent rewrite)	Knows where arguments came from	Holds under adaptive attacks	Keeps task success	Extra model cost	Tamper-evident audit
Injection detectors Lakera Guard, Prompt Guard, Azure Prompt Shields	Yes	No	No (90%+ bypass)	Partial	Low	No
MCP gateways Lasso, Docker MCP Gateway, Pipelock	Yes	Partial	No (content scanning)	Yes	Low	Partial
Progent programmable privilege control	Partial (needs policies)	No	Partial	Partial (45% to 27%)	Medium	No
CaMeL Google DeepMind	No (agent rewrite)	Yes	Yes (side channels)	Partial (84% to 77%)	High (~2.8x tokens)	No
MELON ICML 2025	Partial	No	Partial	Partial (80% to 68%)	~2x (every step)	No
AgentShield (ours) intent contracts + provenance	Yes (SDK or MCP proxy)	Yes, through obfuscation	Deterministic core	Target: within 5 pts	Low (only when needed)	Yes, signed + anchored

Compiled from the published papers and product documentation. The AgentShield row lists design goals that we will measure, not results.

AgentShield: three ideas that no existing tool combines

The agent can think anything. It can only do what the user actually asked for.

1 Intent contracts

Before the agent reads anything untrusted, the user's request is compiled into a signed, frozen list of allowed actions. Injected text can't widen it.

Fixes: hand-written policies, and policy LLMs that attackers target mid-task

2 Argument provenance

Every value in a tool call is traced to its source, through base64, hex, homoglyphs and spelled-out addresses. Secret data can't flow to a public sink.

Fixes: the open problem named in 2026 research; detectors that can be reworded around

3 Risk-adaptive causal check

Only for high-risk calls the rules can't settle, we replay the step with untrusted content masked. If the action disappears, the injection caused it.

Fixes: MELON-style checks that double the cost of every step

Explainable approvals

each pause shows the exact rule and data source, so reviewers can judge it fast

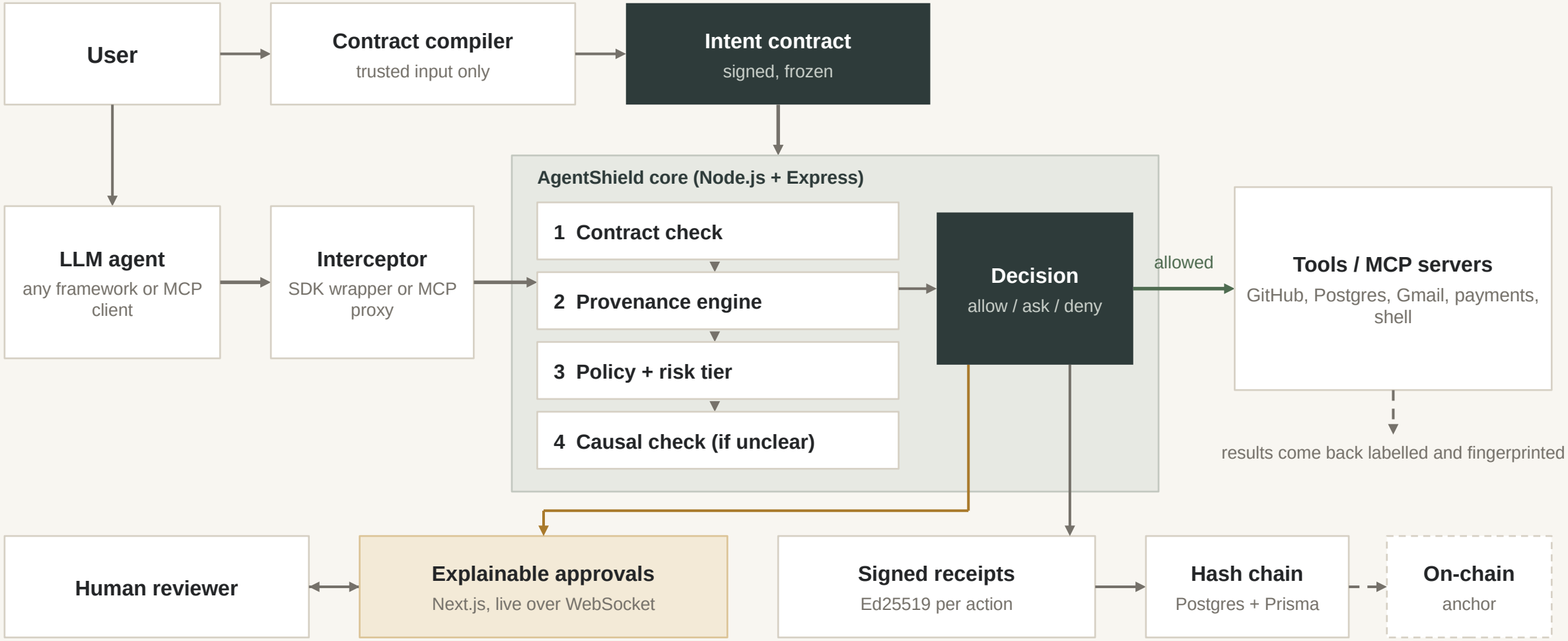
Signed action receipts

hash-chained and anchored on-chain for audits and incident reports

Drop-in

TypeScript SDK, or an MCP proxy for Claude Desktop, Cursor and VS Code

System architecture



How we will prove it

Static benchmarks plus adaptive attacks, because static numbers alone overstate security.

AgentDojo
97 tasks, 629 injection cases (NeurIPS 2024)

InjecAgent
1,054 cases across 17 user tools (ACL Findings 2024)

MCPTox
1,312 tool-poisoning cases on 45 real MCP servers

Adaptive suite (ours)
obfuscated values, contract spoofing, approval social engineering, abuse of already-allowed actions

Metric	Best published reference	AgentShield target
Attack success rate	0.24% (MELON, static attacks only)	Under 1%, also under adaptive attacks
Task success under attack	69% undefended, 59% with MELON	65% or more
Extra model cost	~2x (MELON), ~2.8x tokens (CaMeL)	1.2x or less
Human approvals	not reported	Under 5% of tool calls
Decision latency	not reported	Under 50 ms (rule layers)

References: AgentDojo, GPT-4o (MELON paper); CaMeL (SaTML 2026). Targets will be measured and reported in the next round.

Why it matters, especially in India

Banks and fintech

Agents that move money need least privilege and an audit trail regulators accept.

Software teams

Coding agents hold tokens to private code and production systems.

Compliance

DPDP Rules 2025: penalties up to ₹250 crore for failing reasonable security safeguards. CERT-In: incidents reported within 6 hours. Signed receipts are the evidence.

```
import { Shield } from "agentshield";
const shield = await Shield.fromRequest(userPrompt);
const agent = shield.wrap(myAgent);
// or for MCP clients:  npx agentshield proxy
```

Planned developer API

Thank you

`teamCookie()`

The agent can think anything. It can only do what you asked for.